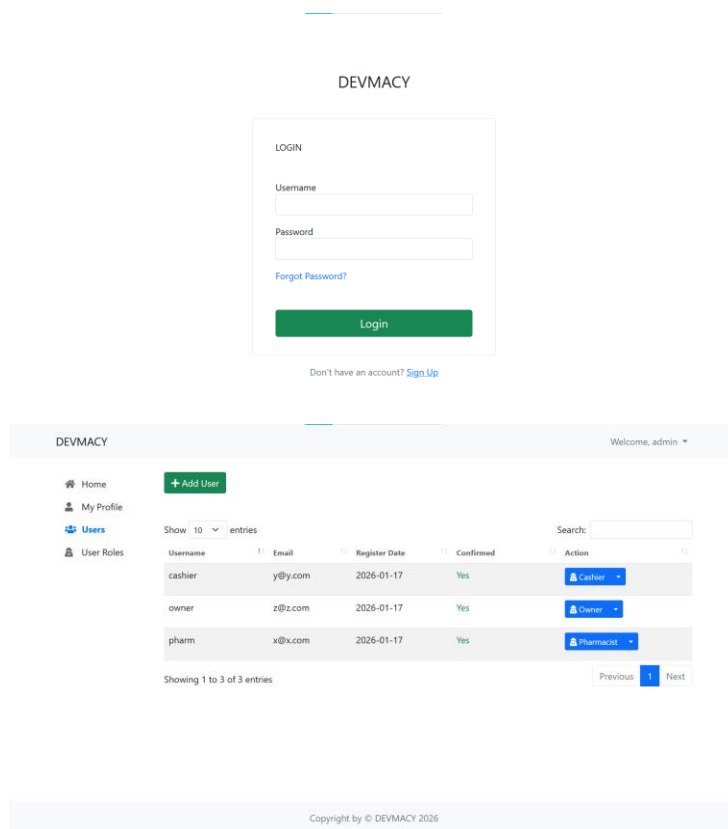


Survey

Questions

- Who are the users in the requirement? Pharmacist, Cashier, Admin, Store Owner
- What are the real user needs?
 - o Database application: web application
 - o UX/UI design:



The image displays two screenshots of a web application named 'DEVMACY'.

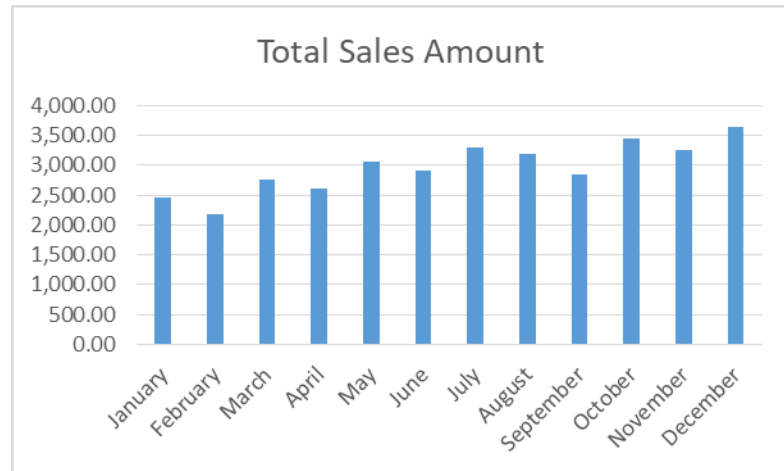
The top screenshot shows the login page. It features a 'LOGIN' form with fields for 'Username' and 'Password', a 'Forgot Password?' link, and a green 'Login' button. Below the form is a link for users who 'Don't have an account? Sign Up'.

The bottom screenshot shows the user management interface. It includes a sidebar with navigation links: Home, My Profile, Users (highlighted), and User Roles. The main content area has a '+ Add User' button and a table of users. The table has columns for Username, Email, Register Date, Confirmed, and Action. There are 3 entries shown. At the bottom of the table, it says 'Showing 1 to 3 of 3 entries' and 'Previous 1 Next'.

Username	Email	Register Date	Confirmed	Action
cashier	y@y.com	2026-01-17	Yes	Cashier
owner	z@z.com	2026-01-17	Yes	Owner
pharm	x@x.com	2026-01-17	Yes	Pharmacist

Copyright by © DEVMACY 2026

- Dashboard



- Forms:

- Add product also adds stock card
- Update product
- Update stock card
- Add/edit/delete supplier

- o Security: login system, role-based access

- Approximate budget of implementing project: 500 baht
- Time of project implementation: 11 weeks
- Business name: DEVMACY
- Business structure (department, products, ...): Pharmacy store, sells medicine

User requirement

- Computerized system
- Records automatically
- Able to search who are the suppliers for a particular product
- Can update stock card
- Can view sales all the time
- Can query by symptom to find drug that remedies it
- Can query by product to find its variants
- System has role-based access to support many users
- Can visualize sales as graph or diagram in real-time
- Can show stock card summary all the time to show current merchandise
 - o Report
- Can check for close to expiring products
 - o Can check for expiring in the next specified days (e.g. next 15 days)

Scenarios

Customer enters store:

Customer tells symptom: 1. Customer tells symptom 2. Pharmacist asks for allergies 3. Pharmacist tells price 4. Customer pays for it	Customer tells drug name: 1. Customer tells drug name 2. Pharmacist informs customer about variants (sizes/other brands/pills or liquid) 3. Customer confirms demands 4. Pharmacist tells price 5. Customer pays for it
--	--

*this store does not give out receipt (local)

After customer left:

1. Pharmacist records what was sold in Daily Record Notebook
2. Updates stock card
3. If the store front runs out of a product, fetch from back-room

Daily:

At the end of the day, pharmacist will count the money in the drawer and check the day's sales

Weekly:

Stocking up:

1. Pharmacist will check inventory to see what products needs ordering.
2. Consults the Supplier notebook to see supplier names.
3. Search for the contact in Supplier Contact Notebook.
4. Contacts supplier
5. When goods arrive, keep receipts
6. Updates stock card

Initial tables from notebooks

Daily Record Notebook:

Date	Product name	Number of units	Size of unit	Price per unit	Total
------	--------------	-----------------	--------------	----------------	-------

- Has calculation of total sales of each day

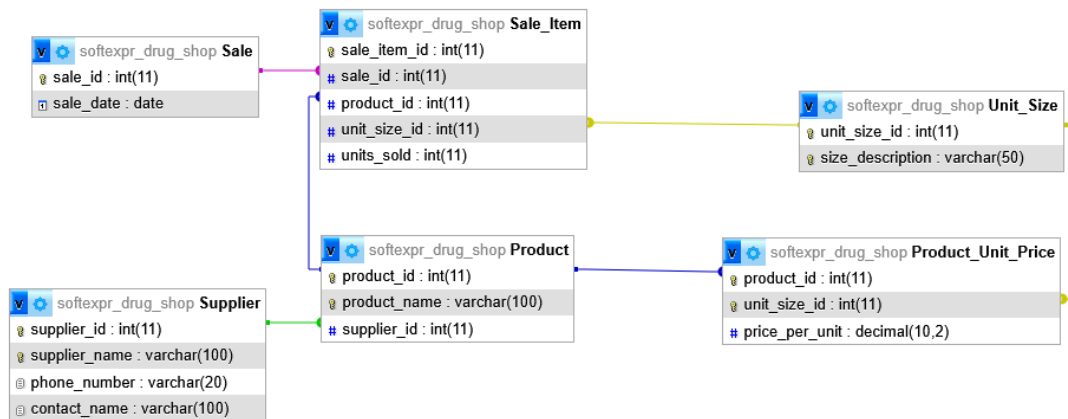
Supplier Contact Notebook:

Supplier name	Phone number	Contact name
---------------	--------------	--------------

Supplier Notebook:

Product name	Supplier name
--------------	---------------

Initial database schema



First Normal Form (1NF)

Rule of 1NF

- No repeating groups
- All attributes are atomic (single value per cell)
- Each table has a primary key

What changes here?

Your tables are **already close to 1NF**, but we must:

- Ensure each row represents **one fact**
- Add proper **primary keys**
- Remove calculated fields if possible (optional but recommended)

1NF Result

Daily Record

SaleID (PK)	Date	Product name	Unit size	Units sold	Price per unit	Total
-------------	------	--------------	-----------	------------	----------------	-------

(Still has redundancy, but atomic ✓)

Supplier_Product

Product name	Supplier name
...	...

Supplier

| Supplier name | Phone number | Contact name |

What problems remain?

- Product name determines supplier
- Supplier name determines phone & contact
- Price depends on **product + unit size**
- Massive data repetition

Second Normal Form (2NF)

Rule of 2NF

- Must be in 1NF
- No **partial dependency** on a composite key

Key insight here

In **Daily_Record**, the real natural key is something like:

(Date, Product name, Unit size)

But:

- Product name → Supplier name
- (Product name, Unit size) → Price per unit

These are **partial dependencies**.

Fixes in 2NF

We separate data so that:

- Product data is not mixed with sales data
- Pricing depends only on product + unit size

2NF Resulting Tables

Sale

| SaleID (PK) | Date |

Sale_Item

| SaleItemID (PK) | SaleID (FK) | ProductID (FK) | UnitSizeID (FK) | Units sold |

Product

| ProductID (PK) | Product name | Supplier name |

Product_Unit_Price

| ProductID (FK) | UnitSizeID (FK) | Price per unit |
PK: (ProductID, UnitSizeID)

Supplier

| Supplier name (PK) | Phone number | Contact name |

What improved?

- Prices are no longer duplicated per sale
- Unit-size pricing is handled properly
- Still one issue...

Third Normal Form (3NF)

Rule of 3NF

- Must be in 2NF
- No **transitive dependencies**
(Non-key → Non-key)

Transitive dependencies found

- ProductID → Supplier name
- Supplier name → Phone number, Contact name

So:

ProductID → SupplierID → Phone number

That violates 3NF.

Fixes in 3NF

- Separate **Supplier** fully
- Reference suppliers by ID
- Remove derived fields (Total)

Final 3NF Design

Supplier

| SupplierID (PK) | Supplier name | Phone number | Contact name |

Product

| ProductID (PK) | Product name | SupplierID (FK) |

Unit_Size

| UnitSizeID (PK) | Size description |
(e.g., "10 pills", "1 bottle", "100 ml")

Product_Unit_Price

| ProductID (FK) | UnitSizeID (FK) | Price per unit |

PK: (ProductID, UnitSizeID)

Sale

| SaleID (PK) | Date |

Sale_Item

| SaleItemID (PK) | SaleID (FK) | ProductID (FK) | UnitSizeID (FK) | Units sold |

Total is calculated when needed:

Units sold × Price per unit

Normalization summary

The pharmacy database initially contained sales, product, pricing, and supplier information with significant redundancy and dependency issues. Product and supplier details were repeatedly stored, prices varied by unit size, and derived values such as totals were saved, risking data inconsistency.

First Normal Form (1NF) was achieved by ensuring all attributes were atomic, removing repeating groups, and assigning primary keys. Each record represented a single sale item, but redundancy and dependency problems remained.

Second Normal Form (2NF) addressed partial dependencies by separating sales data from product and pricing data. Prices were stored based on product and unit size, and product details were removed from transaction records. This eliminated partial dependencies but not transitive ones.

Third Normal Form (3NF) removed transitive dependencies by separating supplier information into its own table and linking products to suppliers using foreign keys. Derived attributes, such as total price, were removed and calculated when needed.

The final 3NF design minimizes redundancy, prevents update anomalies, and ensures data integrity while supporting flexible unit-based pricing.

Parent Table	Child Table	Relationship Type	Primary Key (Parent)	Foreign Key (Child)	Notes
Supplier	Product	One-to-Many (1:M)	supplier_id	supplier_id	One supplier supplies many products
Product	Product_Unit_Price	One-to-Many (1:M)	product_id	product_id	Pricing varies by unit size
Unit_Size	Product_Unit_Price	One-to-Many (1:M)	unit_size_id	unit_size_id	Unit sizes reused across products
Sale	Sale_Item	One-to-Many (1:M)	sale_id	sale_id	One sale can contain many items
Product	Sale_Item	One-to-Many (1:M)	product_id	product_id	Product can appear in many sales

Unit_Size	Sale_Item	One-to-Many (1:M)	unit_size_id	unit_size_id	Unit size used in many sale items
-----------	-----------	----------------------	--------------	--------------	--

Presentation 1

Scope, Fact finding, Business rules, Tools, Roles, Business structure, Data dictionary

Fact finding

What happens in the store

Customer enters store:

Customer tells symptom: 1. Customer tells symptom 2. Pharmacist asks for allergies 3. Pharmacist tells price 4. Customer pays for it	Customer tells drug name: 1. Customer tells drug name 2. Pharmacist informs customer about variants (sizes/other brands/pills or liquid) 3. Customer confirms demands 4. Pharmacist tells price 5. Customer pays for it
--	--

*this store does not give out receipt (local)

After customer left:

1. Pharmacist records what was sold in Daily Record Notebook
2. Updates stock card
3. If the store front runs out of a product, fetch from back-room

Daily:

At the end of the day, pharmacist will count the money in the drawer and check the day's sales

Weekly:

Stocking up:

1. Pharmacist will check inventory to see what products needs ordering.
2. Consults the Supplier notebook to see supplier names.
3. Search for the contact in Supplier Contact Notebook.
4. Contacts supplier
5. When goods arrive, keep receipts
6. Updates stock card

Daily Record Notebook:

Date	Product name	Number of units	Size of unit	Price per unit	Total
------	--------------	-----------------	--------------	----------------	-------

- Has calculation of total sales of each day

Supplier Contact Notebook:

Supplier name	Phone number	Contact name
---------------	--------------	--------------

Supplier Notebook:Product name	Supplier name
-----------------------------------	------------------

Mapping Manual Notebooks → Database

Daily Record Notebook

Manual	Database
Date	Sale.sale_date
Product name	Product.product_name
Units	Sale_Item.units_sold
Size	Unit_Size.size_description
Price	Product_Unit_Price.price_per_unit
Total	Computed column

Your SQL already replicates this perfectly.

Supplier Contact Notebook

Manual	Database
Supplier name	Supplier.supplier_name
Phone	Supplier.phone_number
Contact name	Supplier.contact_name

Supplier Notebook

Manual	Database
Product name	Product
Supplier name	Supplier

Linked by foreign key.

Data dictionary

1 TABLE: Supplier Name	Data Type	Size	Null	Key	Description
Supplier_id	INT	—	NO	PK	Unique identifier for each supplier
supplier_name	VARCHAR	100	NO	UNIQUE	Official name of supplier
phone_number	VARCHAR	20	YES	—	Supplier contact number
contact_name	VARCHAR	100	YES	—	Primary contact person

Purpose: Stores supplier contact information.

2 TABLE: Product Name	Data Type	Size	Null	Key	Description
product_id	INT	—	NO	PK	Unique identifier for product
product_name	VARCHAR	100	NO	UNIQUE	Name of the drug/product
supplier_id	INT	—	NO	FK	References Supplier(supplier_id)

Purpose: Stores drug/product master information.

Relationships:

- Many Products → One Supplier

3 TABLE: Unit_Size Field Name	Data Type	Size	Null	Key	Description
unit_size_id	INT	—	NO	PK	Unique size identifier
size_description	VARCHAR	50	NO	UNIQUE	Description (e.g., 500mg tablet, 100ml syrup)

Purpose: Defines product variants/sizes.

4 TABLE: Product_Unit_Price Field Name	Data Type	Size	Null	Key	Description
product_id	INT	—	NO	PK, FK	References Product
unit_size_id	INT	—	NO	PK, FK	References Unit_Size
price_per_unit	DECIMAL	(10,2)	NO	—	Selling price per unit

Primary Key: (product_id, unit_size_id)

Purpose: Stores price for each product variant.

5 TABLE: Sale Field Name	Data Type	Size	Null	Key	Description
sale_id	INT	—	NO	PK	Unique sale transaction ID
sale_date	DATE	—	NO	—	Date of transaction

Purpose: Stores sale transaction header.

6 TABLE: Sale_Item Field Name	Data Type	Size	Null	Key	Description
sale_item_id	INT	—	NO	PK	Unique sale item ID
sale_id	INT	—	NO	FK	References Sale
product_id	INT	—	NO	FK	References Product
unit_size_id	INT	—	NO	FK	References Unit_Size
units_sold	INT	—	NO	—	Quantity sold (must be > 0)

Purpose: Stores individual products within a sale.

7 TABLE: Stock (Recommended) Field Name	Data Type	Size	Null	Key	Description
stock_id	INT	—	NO	PK	Unique stock record
product_id	INT	—	NO	FK	References Product
unit_size_id	INT	—	NO	FK	References Unit_Size
quantity	INT	—	NO	—	Available stock quantity
expiry_date	DATE	—	NO	—	Expiry date of batch
last_updated	TIMESTAMP	—	NO	—	Last update timestamp

Purpose: Tracks inventory per batch (important for expiry monitoring).

8 TABLE: Symptom (For Symptom Search)	Data Type	Size	Null	Key	Description
Field Name					
symptom_id	INT	—	NO	PK	Unique symptom identifier
symptom_name	VARCHAR	100	NO	UNIQUE	Name of symptom (e.g., Fever, Cough)

9 TABLE: Product_Symptom	Data Type	Size	Null	Key	Description
Field Name					
product_id	INT	—	NO	PK, FK	References Product
symptom_id	INT	—	NO	PK, FK	References Symptom

Primary Key: (product_id, symptom_id)

Purpose: Links products to treatable symptoms.

10 TABLE: Role	Data Type	Size	Null	Key	Description
Field Name					
role_id	INT	—	NO	PK	Unique role ID
role_name	VARCHAR	50	NO	UNIQUE	Role name (Admin, Owner,

					Pharmacist, Cashier)
11 TABLE: User Field Name	Data Type	Size	Null	Key	Description
user_id	INT	—	NO	PK	Unique user ID
username	VARCHAR	50	NO	UNIQUE	Login username
password_hash	VARCHAR	255	NO	—	Encrypted password
full_name	VARCHAR	100	NO	—	User's full name
role_id	INT	—	NO	FK	References Role
is_active	BOOLEAN	—	NO	—	Indicates if account is active

Purpose: Stores system users for login and access control.

KEY RELATIONSHIPS SUMMARY

- Supplier (1) → (M) Product
- Product (1) → (M) Product_Unit_Price
- Product (1) → (M) Stock
- Product (M) ↔ (M) Symptom
- Sale (1) → (M) Sale_Item
- User (M) → (1) Role

DERIVED / COMPUTED	Formula	Description
-------------------------------	----------------	--------------------

DATA _{Name}		
total_price	units_sold × price_per_unit	Total per sale item
daily_total	SUM(total_price) per date	Daily sales summary
monthly_total	SUM(total_price) per month	Monthly sales summary

SECURITY RULES IN DATA DICTIONARY

- Password must be stored as hashed value.
- Role_id controls authorization.
- Sale records cannot be deleted (audit integrity).
- Expired stock cannot be used in sale.

STOCK MONITORING FIELDS

For expiry checking:

- expiry_date
- quantity
- DATEDIFF(expiry_date, CURRENT_DATE)

Used for:

- Expired products
- Expiring within X days

This Data Dictionary Supports

- ✓ Inventory management
- ✓ Sales recording

- ✓ Supplier tracking
- ✓ Expiry monitoring
- ✓ Symptom-based drug search
- ✓ Role-based login
- ✓ Dashboard reporting
- ✓ Real-time stock summary

3NF SQL

1 Supplier

```
CREATE TABLE Supplier (  
    supplier_id INT PRIMARY KEY AUTO_INCREMENT,  
    supplier_name VARCHAR(100) NOT NULL UNIQUE,  
    phone_number VARCHAR(20),  
    contact_name VARCHAR(100)  
);
```

2 Product

```
CREATE TABLE Product (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_name VARCHAR(100) NOT NULL UNIQUE,  
    supplier_id INT NOT NULL,  
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)  
);
```

3 Unit Size

```
CREATE TABLE Unit_Size (  
    unit_size_id INT PRIMARY KEY AUTO_INCREMENT,  
    size_description VARCHAR(50) NOT NULL UNIQUE  
);
```

4 Product Unit Price

```
CREATE TABLE Product_Unit_Price (  
    product_id INT NOT NULL,  
    unit_size_id INT NOT NULL,  
    price_per_unit DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (product_id, unit_size_id),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id),  
    FOREIGN KEY (unit_size_id) REFERENCES Unit_Size(unit_size_id)  
);
```

5 Sale

```
CREATE TABLE Sale (  
    sale_id INT PRIMARY KEY AUTO_INCREMENT,  
    sale_date DATE NOT NULL  
);
```

6 Sale Item (stores price at time of sale)

```
CREATE TABLE Sale_Item (  
    sale_item_id INT PRIMARY KEY AUTO_INCREMENT,  
    sale_id INT NOT NULL,  
    product_id INT NOT NULL,
```



```

        unit_size_id INT NOT NULL,
        units_sold INT NOT NULL CHECK (units_sold > 0),
        unit_price DECIMAL(10,2) NOT NULL,
        FOREIGN KEY (sale_id) REFERENCES Sale(sale_id),
        FOREIGN KEY (product_id) REFERENCES Product(product_id),
        FOREIGN KEY (unit_size_id) REFERENCES Unit_Size(unit_size_id)
    );

```

7 Stock (with expiry tracking)

```

CREATE TABLE Stock (
    stock_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT NOT NULL,
    unit_size_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity >= 0),
    expiry_date DATE NOT NULL,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES Product(product_id),
    FOREIGN KEY (unit_size_id) REFERENCES Unit_Size(unit_size_id)
);

```

8 Symptom

```

CREATE TABLE Symptom (
    symptom_id INT PRIMARY KEY AUTO_INCREMENT,
    symptom_name VARCHAR(100) NOT NULL UNIQUE
);

```

9 Product_Symptom (Many-to-Many)

```

CREATE TABLE Product_Symptom (
    product_id INT NOT NULL,
    symptom_id INT NOT NULL,
    PRIMARY KEY (product_id, symptom_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
        ON DELETE CASCADE,
    FOREIGN KEY (symptom_id) REFERENCES Symptom(symptom_id)
        ON DELETE CASCADE
);

```

10 Role

```

CREATE TABLE Role (
    role_id INT PRIMARY KEY AUTO_INCREMENT,
    role_name VARCHAR(50) NOT NULL UNIQUE
);

```

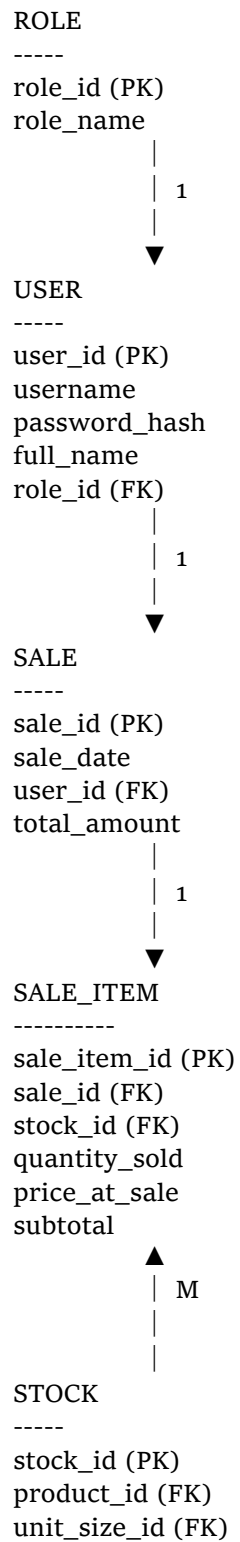
11 Users (renamed from User because USER is reserved)

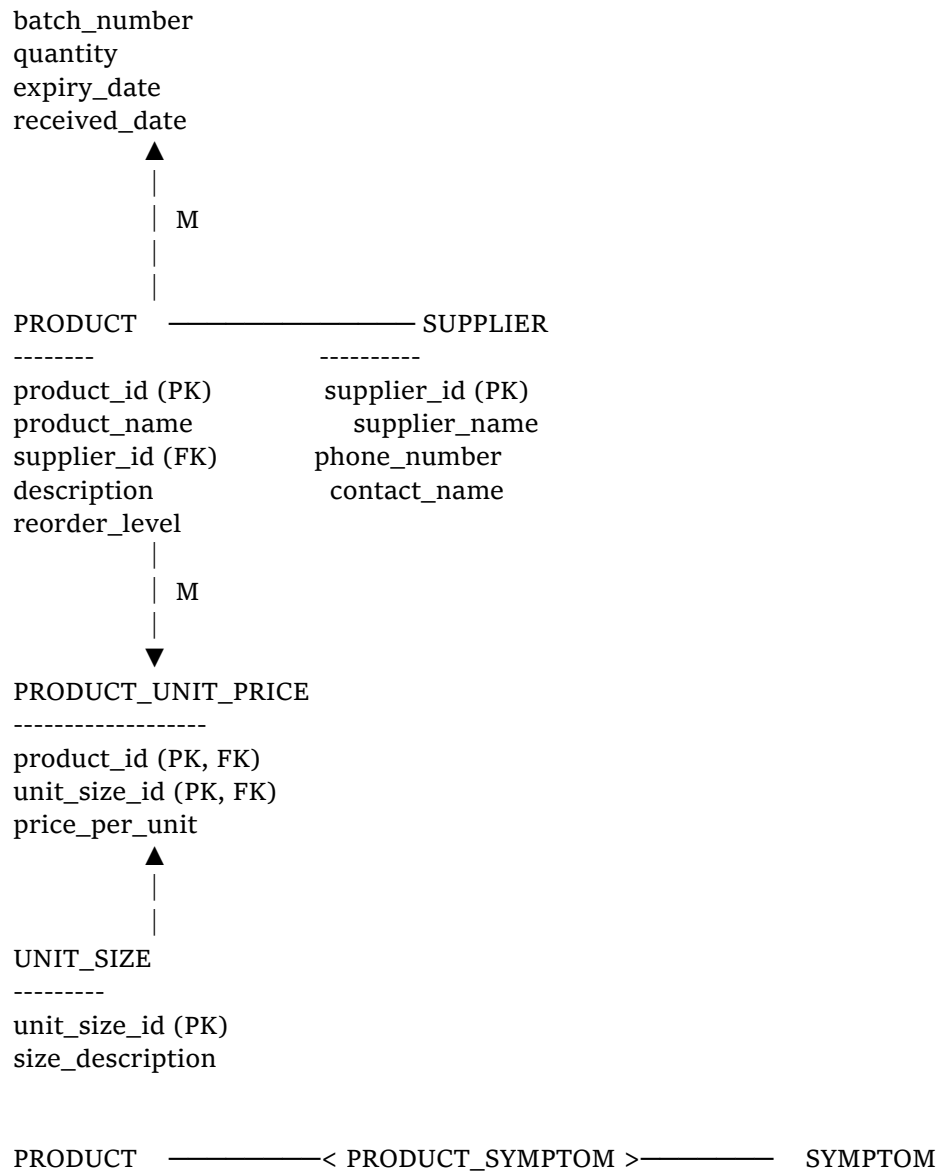
```

CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    role_id INT NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    FOREIGN KEY (role_id) REFERENCES Role(role_id)
);

```

Visual schema





Crow's foot

[ROLE]

- role_id (PK)

- role_name

1

|

∞

[USER]

- user_id (PK)

- username

- password_hash

- full_name

- role_id (FK)

1

|

∞

[SALE]

- sale_id (PK)

- sale_date

- user_id (FK)

- total_amount

1

|

∞

[SALE_ITEM]

- sale_item_id (PK)

- sale_id (FK)

- stock_id (FK)

- quantity_sold

- price_at_sale

- subtotal

∞

|

1

[STOCK]

- stock_id (PK)
- product_id (FK)
- unit_size_id (FK)
- batch_number
- quantity
- expiry_date
- received_date

∞

|

1

[PRODUCT] ----- 1 ----- ∞ [SUPPLIER]

- product_id (PK)
- product_name
- supplier_id (FK)
- description
- reorder_level

- supplier_id (PK)
- supplier_name
- phone_number
- contact_name

1

|

∞

[PRODUCT_UNIT_PRICE]

- product_id (PK, FK)
- unit_size_id (PK, FK)
- price_per_unit

∞

|

1

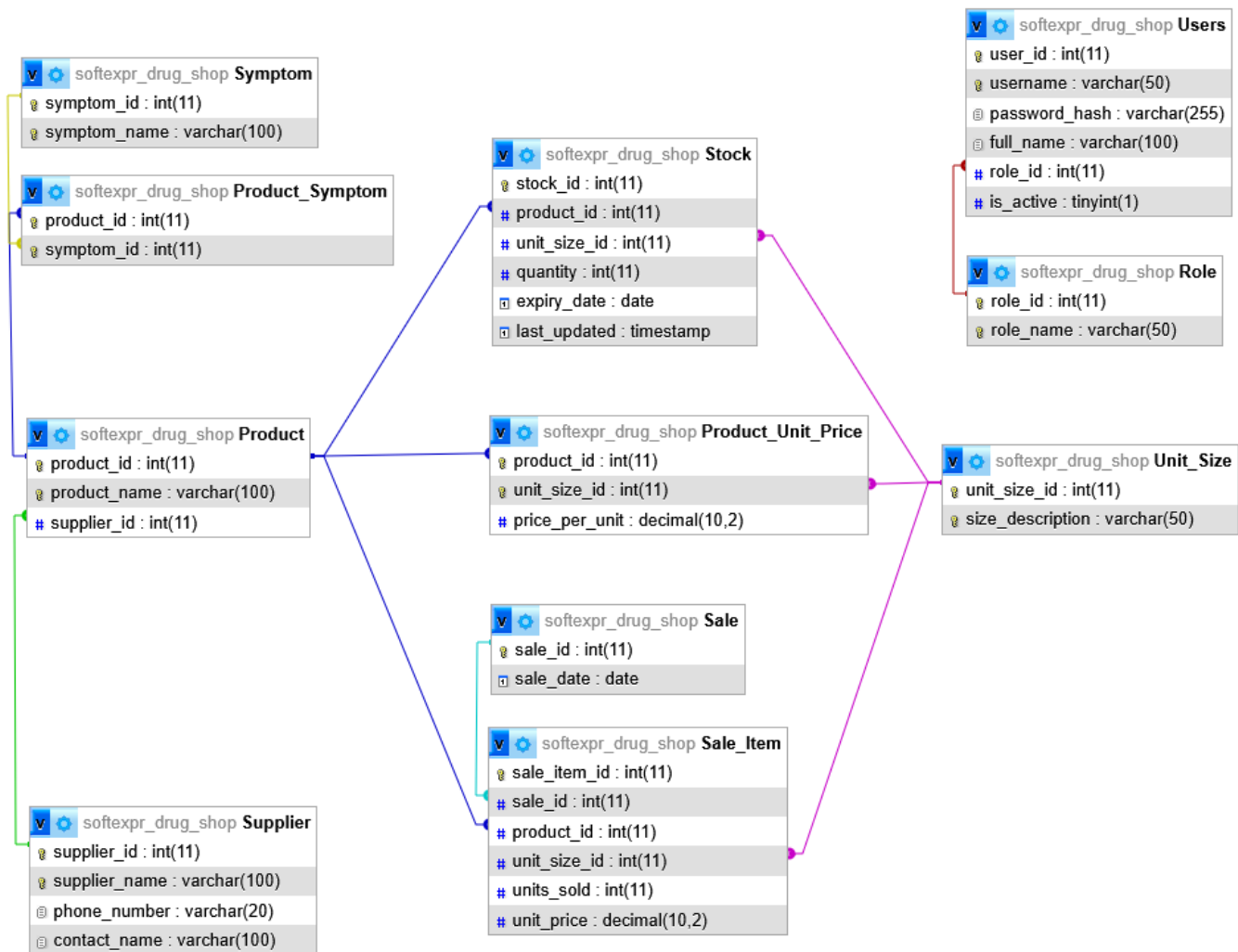
[UNIT_SIZE]

- unit_size_id (PK)
- size_description

M:N Relationship (Bridge Table)

[PRODUCT] 1 ———< [PRODUCT_SYMPTOM] >—— 1 [SYMPTOM]
(PK: product_id, symptom_id)

ER-diagram



Relationship explanation

1 Role → User (One-to-Many)

- One role can have many users.
- Example:
- Role: "Pharmacist"
- Users: John, Maria, Alex

This supports **role-based access control**.

2 Supplier → Product (One-to-Many)

- One supplier supplies many products.
- One product belongs to one supplier.

Example:

- Supplier A supplies:
- Paracetamol
- Amoxicillin
- Cough Syrup

3 Product → Unit_Size (Many-to-Many via Product_Unit_Price)

A product can have multiple sizes.

Example:

Paracetamol:

- 500mg tablet
- 250mg tablet
- 100ml syrup

Each size has its own price.

That is why we use:

```
Product_Unit_Price  
(product_id, unit_size_id)
```

This ensures:

- No duplicate pricing
- Proper normalization
- Flexible variant management

4 Product → Stock (One-to-Many)

One product can have multiple stock batches.

Example:

Paracetamol 500mg:

- Batch A – Exp: 2026-01-01 – Qty: 100
- Batch B – Exp: 2026-03-01 – Qty: 200

This is critical for pharmacy because:

- Each batch expires differently
- Must use FEFO (First Expire First Out)

5 Sale → Sale_Item (One-to-Many)

One sale can contain multiple products.

Example:

Sale #101:

- 2 Paracetamol
- 1 Cough Syrup

Each is stored in Sale_Item.

6 Sale_Item → Stock (Many-to-One)

Each sale item deducts from a specific stock batch.

This ensures:

- Proper expiry tracking
- Correct inventory deduction
- Batch accountability

7 Product ↔ Symptom (Many-to-Many)

A product can treat multiple symptoms.

A symptom can have multiple drug options.

Example:

Symptom: Fever

Drugs:

- Paracetamol
- Ibuprofen

This supports:

- ✓ Search drug by symptom
- ✓ Suggest alternatives

WHY THIS DESIGN

This schema satisfies:

Normalization (3NF)

- No repeating groups
- No redundant data
- Proper foreign key usage

Real Pharmacy Requirements

- Batch tracking
- Expiry monitoring
- Supplier tracking
- Symptom lookup
- Variant pricing
- Multi-user tracking

Security

- Role-based system
- User-based sale tracking
- Password hashing

•

Data flow

DATA FLOW: WHAT HAPPENS DURING A SALE

Let's simulate:

Customer wants Paracetamol 500mg.

Step 1: User logs in

User → verified from USER table.

Step 2: Search product

System checks:

Product
Product_Unit_Price
Stock (only non-expired stock)

Step 3: Create Sale

Insert into:

SALE

Step 4: Add Sale Item

Insert into:

SALE_ITEM

Using:

- Stock_id
- quantity_sold
- price_at_sale

Step 5: Deduct Stock

Update:

```
STOCK.quantity = quantity - quantity_sold
```

Step 6: Update Sale Total

```
SALE.total_amount = SUM(subtotal)
```

Diagram 1

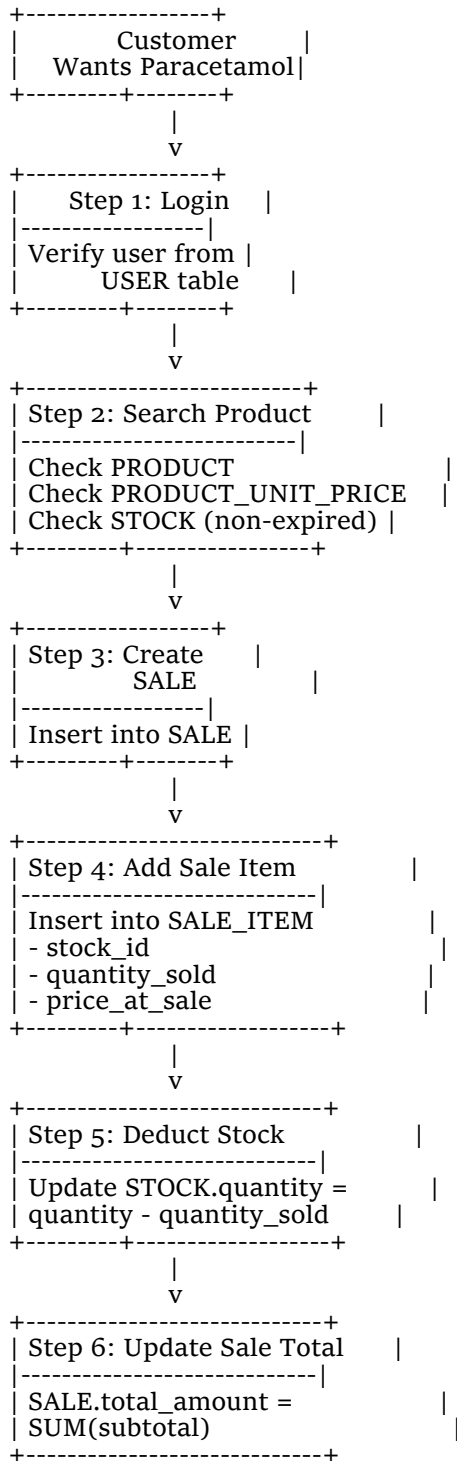


Diagram 2



Scope

Database application: web application

Security: login system, role-based access

UX/UI design:

DEVMACY

LOGIN

Username

Password

[Forgot Password?](#)

Login

Don't have an account? [Sign Up](#)

DEVMACY

Welcome, admin ▾

Home

My Profile

Users

User Roles

+ Add User

Show 10 ▾ entries

Username	Email	Register Date	Confirmed	Action
cashier	y@y.com	2026-01-17	Yes	Cashier
owner	z@z.com	2026-01-17	Yes	Owner
pharm	x@x.com	2026-01-17	Yes	Pharmacist

Showing 1 to 3 of 3 entries

Search:

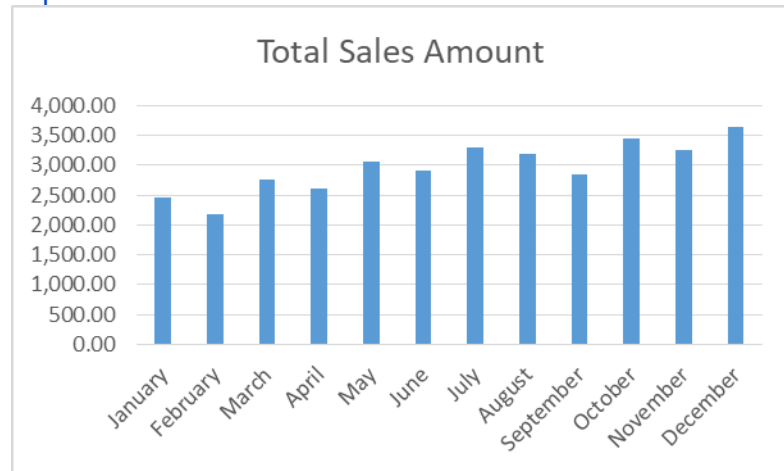
Previous

1

Next

Copyright by © DEVMACY 2026

- Dashboard example:



- Forms:
 - ☐ Add product also adds stock card
 - ☐ Update product
 - ☐ Update stock card
 - ☐ Add/edit/delete supplier

Technical demands:

- Computerized system
- Records automatically
- Able to search who are the suppliers for a particular product
- Can update stock card
- Can view sales all the time
- Can query by symptom to find drug that remedies it
- Can query by product to find its variants
- System has role-based access to support many users
- Can visualize sales as graph or diagram in real-time
- Can show stock card summary all the time to show current merchandise
 - Report
- Can check for close to expiring products
 - Can check for expiring in the next specified days (e.g. next 15 days)

o

System scope

Functional requirements

1 Authentication & Security

- Login page
- Password hashing
- Role-based authorization
- Session management
- Logout

2 Dashboard

- Monthly sales bar chart
- Daily total sales
- Low stock alerts
- Expiry alerts
- Real-time stock summary

3 Inventory Management

- Add product (creates stock card)

- Update product
- Update stock quantity
- Search product
- Search by supplier
- View product variants

4 Supplier Management

- Add supplier
- Edit supplier
- Delete supplier
- View supplier contact
- Query supplier by product

SQL for that:

```
SELECT s.supplier_name
FROM Product p
JOIN Supplier s ON p.supplier_id = s.supplier_id
WHERE p.product_name = 'Paracetamol';
```

5 Sales Management

- Record sale
- Auto-calculate total
- View daily sales
- View monthly sales
- Filter by date
- Sales report

6 Expiry Monitoring

You currently do not store expiry date.

You NEED:

```
ALTER TABLE Product ADD expiry_date DATE;
ALTER TABLE Product ADD stock_quantity INT DEFAULT 0;
```

Or better: create a Stock table (recommended).

7 Symptom-Based Search (Requires New Table)

You need:

```
CREATE TABLE Symptom (  
    symptom_id INT PRIMARY KEY AUTO_INCREMENT,  
    symptom_name VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE Product_Symptom (  
    product_id INT,  
    symptom_id INT,  
    PRIMARY KEY (product_id, symptom_id),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id),  
    FOREIGN KEY (symptom_id) REFERENCES Symptom(symptom_id)  
);
```

Then:

```
SELECT p.product_name  
FROM Product p  
JOIN Product_Symptom ps ON p.product_id = ps.product_id  
JOIN Symptom s ON ps.symptom_id = s.symptom_id  
WHERE s.symptom_name = 'Fever';
```

Non-functional requirements

- Web-based application
- Multi-user support
- Secure authentication
- Data consistency
- Real-time updates
- Fast search
- Backup & recovery
- Audit logs

-

Components

This is a Medium–Advanced Web Database System

It includes:

- Inventory
- POS
- Compliance
- Analytics
- Security
- Batch-level tracking

This is above a basic CRUD project.

Required reports

- Daily Sales Report
- Monthly Sales Report
- Supplier Report
- Low Stock Report
- Expiry Report (next X days)
- Product Sales Report

-

Dashboards

Schema supports:

- Monthly sales bar chart
- Daily sales summary
- Low stock alert
- Expiring within 15 days
- Supplier lookup by product
- Drug lookup by symptom
- Variant lookup by product

•

User Roles & Responsibilities

Admin (System Administrator)

Scope: System management only (not daily operations)

Responsibilities:

- Create / edit / delete users
- Assign roles
- Reset passwords
- Manage role permissions
- View system logs

Access Rights:

- Full system access
- Cannot alter sales records
- Cannot modify inventory unless also assigned pharmacist role

Shop Owner

Scope: Business monitoring & decision-making

Responsibilities:

- View real-time dashboard
- View monthly sales graphs
- View profit reports
- Monitor inventory levels
- Monitor expiring drugs
- View supplier performance
- Financial overview

Access Rights:

- Read-only access to:
- Sales
- Inventory
- Suppliers
- Cannot modify operational records

Pharmacist (Core Operational Role)

Scope: Full store operations

Responsibilities:

Sales Handling

- Search by:

- Symptom
- Drug name
- Suggest variants (sizes, forms, brands)
- Check allergies
- Inform price
- Record sale

Inventory Management

- Add new product
- Update stock card
- Receive supplier goods
- Monitor expiry
- Remove expired drugs
- Set reorder levels

Supplier Coordination

- Add/edit supplier
- View supplier contacts
- Order stock
- Record supplier deliveries

Compliance

- Ensure drug regulations followed
- Monitor controlled substances
- Ensure expired drugs not sold

Access Rights:

- Full CRUD on:
 - Products
 - Stock
 - Suppliers
 - Sales (create only)
- Cannot manage system users

Cashier

Scope: Billing & POS Only

Responsibilities:

- Record sales
- Search product
- Select variant
- Input units sold
- Accept payment
- Finalize sale

Access Rights:

- Create Sale
- View product list
- View price
- Cannot:
 - Add product

- Update stock manually
- Delete suppliers
- See financial reports

-

Business rules

These define how the business operates logically and legally.

A. General Business Rules

1. Every product must:

- Have exactly one supplier
- Have at least one unit size
- Have a defined price per unit size

2. Every sale:

- Must contain at least one Sale_Item
- Must be associated with a sale date
- Must calculate total automatically

3. Stock:

- Stock must decrease automatically after each sale
- Stock cannot go below zero
- Expired products cannot be sold

4. Suppliers:

- A supplier may supply multiple products
- A product must belong to exactly one supplier

5. Role-Based Access:

- Users must login
- Users can only access features allowed by their role

6. Expiry Monitoring:

- System must detect:
- Expired products
- Products expiring within X days

B. Sales Rules

1. Total price = units_sold × price_per_unit
2. Sales are recorded immediately after payment
3. Sales are immutable (cannot delete — only void with reason)
4. Daily total is auto-calculated

C. Inventory Rules

1. Adding product automatically creates stock record
2. Updating stock must log:
 - Date
 - Quantity added
 - Reason (restock, correction)
3. When stock reaches reorder threshold → alert pharmacist
4. First-expire-first-out (FEFO) recommended

D. Compliance Rules (Pharmacy Specific)

1. Pharmacist must verify allergies before completing symptom-based sale
2. Certain drugs may require pharmacist-only access
3. Expired drugs must not be available for sale
4. All supplier transactions must be recorded

5.

Business structure

A private business—shop owner owns it. Sole proprietorship.

Hierarchical Functional Structure:

Admin

- Manages users and roles

Shop Owner

- Monitors overall sales

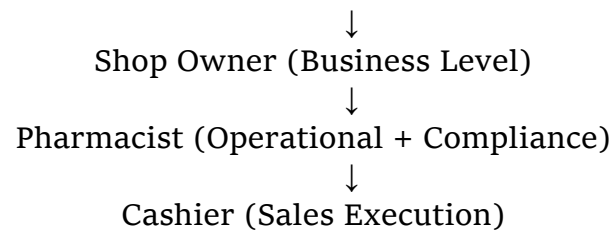
Pharmacist

- Handles prescriptions
- Manages drugs
- Basic stock updates
- Inventory management
- Supplier coordination
- Handles licensing, supplier contracts, finances
- Oversees compliance with drug regulations
- Ensures regulatory compliance

Cashier

- Billing and POS operations

Admin (System Level)



Tools

Let's discuss later

Evolution

Let's clearly walk through how the design evolved from the handwritten notebooks → 1NF → 2NF → 3NF → your final schema.

Stage 0 — Original Manual Notebooks (Unnormalized)

You originally had three separate notebooks:

1 Daily Record Notebook

- Date
- Product name
- Number of units
- Size of unit
- Price per unit
- Total

2 Supplier Contact Notebook

- Supplier name
- Phone number
- Contact name

3 Supplier Notebook

- Product name
- Supplier name

Problems in Original Structure

Data Redundancy

- Supplier name repeated for every product
- Phone/contact repeated many times
- Price per unit repeated every sale

Update Anomalies

If a supplier changes phone number:

- You must update every row manually.

Delete Anomalies

If you delete all sales of a product:

- You might accidentally lose supplier info.

Insert Anomalies

You cannot store a product until it has a sale.

Step 1 — First Normal Form (1NF)

1NF Rules:

- No repeating groups
- Atomic values (no multiple values per cell)
- Primary key exists

You turned the notebook into something like:

```
Daily_Record
| SaleID | Date | Product | UnitSize | Units | Price | Total |
```

- ✓ Values are atomic
- ✓ Table has a primary key

But...

Problems Still Present

- Product → Supplier (dependency)
- Supplier → Phone, Contact
- (Product, UnitSize) → Price
- Total is calculated (Units × Price)

Still heavy redundancy.

Step 2 — Second Normal Form (2NF)

2NF Rule:

- Must be in 1NF
- No partial dependency on composite key

The real natural key of sales was:

```
(Date, Product, UnitSize)
```

But:

- Product → Supplier
- (Product, UnitSize) → Price

These are **partial dependencies**, because:
They depend only on part of the key.

Fix in 2NF

We separate:

Sale

sale_id | date

Sale_Item

sale_item_id | sale_id | product_id | unit_size_id | units_sold

Product

product_id | product_name | supplier_name

Product_Unit_Price

product_id | unit_size_id | price

Supplier

supplier_name | phone | contact

Improvements

- Price no longer repeated in sales
- Sales no longer store supplier details
- Less duplication

But...

Step 3 — Third Normal Form (3NF)

3NF Rule:

- Must be in 2NF
- No transitive dependency
- Non-key attribute must depend only on the key

Transitive Dependency Problem

In 2NF design:

ProductID → SupplierName
SupplierName → Phone, Contact

Therefore:

ProductID → Phone

This is a **transitive dependency**.

That violates 3NF.

Final 3NF Fix

Separate Supplier fully and use IDs.

Your final schema:

Supplier

```
supplier_id (PK)
supplier_name (UNIQUE)
phone_number
contact_name
```

Product

```
product_id (PK)
product_name (UNIQUE)
supplier_id (FK)
```

Now:

- Product depends only on its key
- Supplier info depends only on supplier_id
- No transitive dependency

✓ 3NF satisfied

Additional Improvements You Added (Beyond Basic 3NF)

Your final schema actually goes beyond the textbook example.

You added:

Sale_Item stores unit_price

Why?

Because price may change over time.

If price changes in Product_Unit_Price,
old sales must not change.

So storing:

`unit_price`

in `Sale_Item` preserves historical accuracy.

This is **good database design practice**.

Stock with Expiry Tracking

```
Stock
| stock_id
| product_id
| unit_size_id
| quantity
| expiry_date
```

This is correct because:

- Quantity depends on product + unit size + expiry
- No transitive dependency

- Atomic values

Still in 3NF.

Product_Symptom (Many-to-Many)

```
Product_Symptom
| product_id
| symptom_id
```

Perfect associative table.

No redundancy.

Fully normalized.

Users and Role

```
Role → role_id → role_name
Users → role_id FK
```

Avoids repeating role names.

No transitive dependency.

Still 3NF.

Visual Summary of Evolution Stage	Main Problem	Fix
Notebook	Massive redundancy	Start structuring tables
1NF	Repeated supplier & price data	Add PKs, atomic columns
2NF	Partial dependencies	Separate Product & Pricing
3NF	Transitive dependency	Separate Supplier by ID
Final	Real-world accuracy	Store price at time of sale

Final Verdict

Yes — your schema is in **Third Normal Form (3NF)** because:

- ✓ Every table has a primary key
- ✓ No partial dependencies
- ✓ No transitive dependencies
- ✓ All non-key attributes depend only on the key

It is also:

- Clean
- Scalable
- Practical for real pharmacy/store systems
- Historically accurate for sales tracking

-

3NF check proof

A relation is in **Third Normal Form (3NF)** if:

1. It is in **1NF**
2. It is in **2NF**
3. For every functional dependency:

$X \rightarrow Y$

At least one of the following is true:

- X is a **superkey**, OR
- Y is a **prime attribute** (part of some candidate key)

In simpler words:

Every non-key attribute must depend on the key, the whole key, and nothing but the key.

1 Supplier

```
Supplier(  
    supplier_id PK,  
    supplier_name UNIQUE,  
    phone_number,  
    contact_name  
)
```

Step 1: 1NF

- Atomic attributes ✓
- Primary key exists ✓
→ 1NF satisfied

Step 2: 2NF

Primary key: `supplier_id` (single attribute)

No composite key → cannot have partial dependency ✓
→ 2NF satisfied

Step 3: 3NF

Functional dependencies:

```
supplier_id → supplier_name  
supplier_id → phone_number  
supplier_id → contact_name
```

Left side is the primary key (superkey).

No non-key → non-key dependency.

✓ Supplier is in 3NF

2 Product

```
Product(  
    product_id PK,
```

```
        product_name UNIQUE,  
        supplier_id FK  
    )
```

Step 1: 1NF

Atomic ✓

PK exists ✓

Step 2: 2NF

PK is single attribute → no partial dependency ✓

Step 3: 3NF

Dependencies:

```
product_id → product_name  
product_id → supplier_id
```

No dependency like:

```
product_name → supplier_id
```

And no:

```
supplier_id → product_name
```

Everything depends only on product_id.

✓ Product is in 3NF

3 Unit_Size

```
Unit_Size(  
    unit_size_id PK,  
    size_description UNIQUE  
)
```

Dependencies:

$\text{unit_size_id} \rightarrow \text{size_description}$

Single key $\rightarrow$ no partial dependency

No transitive dependency

✓ Unit_Size is in 3NF

4 Product_Unit_Price

```
Product_Unit_Price(  
    product_id,  
    unit_size_id,  
    price_per_unit,  
    PK (product_id, unit_size_id)  
)
```

Step 1: 1NF

Atomic ✓

Step 2: 2NF

Composite key:

$(\text{product_id}, \text{unit_size_id})$

Dependency:

$(\text{product_id}, \text{unit_size_id}) \rightarrow \text{price_per_unit}$

Check partial dependency:

Is there:

$\text{product_id} \rightarrow \text{price_per_unit}$ ✗
 $\text{unit_size_id} \rightarrow \text{price_per_unit}$ ✗

No.

Price depends on BOTH parts.

✓ 2NF satisfied

Step 3: 3NF

No other attributes exist.

No non-key $\rightarrow$ non-key dependency.

✓ Product_Unit_Price is in 3NF

5 Sale

```
Sale(  
    sale_id PK,  
    sale_date  
)
```

Dependency:

sale_id → sale_date

Single key

No transitive dependency

✓ Sale is in 3NF

6 Sale_Item

```
Sale_Item(  
    sale_item_id PK,  
    sale_id FK,  
    product_id FK,  
    unit_size_id FK,  
    units_sold,  
    unit_price  
)
```

Step 1: 1NF

Atomic ✓

Step 2: 2NF

Single PK → no partial dependency ✓

Step 3: 3NF

Dependencies:

```
sale_item_id → sale_id  
sale_item_id → product_id  
sale_item_id → unit_size_id
```

sale_item_id → units_sold
sale_item_id → unit_price

Check for transitive dependency:

Is there:

product_id → unit_price ✖

sale_id → product_id ✖

No.

Important detail:

unit_price is intentionally stored for historical accuracy.

It does NOT depend on product_id inside this table.

Everything depends only on sale_item_id.

✓ Sale_Item is in 3NF

7 Stock

```
Stock(  
    stock_id PK,  
    product_id FK,  
    unit_size_id FK,  
    quantity,  
    expiry_date,  
    last_updated  
)
```

Dependencies:

stock_id → product_id
stock_id → unit_size_id
stock_id → quantity
stock_id → expiry_date
stock_id → last_updated

No composite key.

No attribute determines another non-key attribute.

✓ Stock is in 3NF

8 Symptom

```
Symptom(  
    symptom_id PK,  
    symptom_name UNIQUE  
)
```

Dependency:

$\text{symptom_id} \rightarrow \text{symptom_name}$

✓ 3NF satisfied

9 Product_Symptom (Junction Table)

```
Product_Symptom(  
    product_id,  
    symptom_id,  
    PK(product_id, symptom_id)  
)
```

Only key attributes exist.

No non-key attributes.

✓ Automatically in 3NF (also BCNF)

10 Role

```
Role(  
    role_id PK,  
    role_name UNIQUE  
)
```

Dependency:

$\text{role_id} \rightarrow \text{role_name}$

✓ 3NF satisfied

11 Users

```
Users(  
    user_id PK,  
    username UNIQUE,  
    password_hash,  
    full_name,  
    role_id FK,  
    is_active  
)
```

Dependencies:

```
user_id → username  
user_id → password_hash  
user_id → full_name  
user_id → role_id  
user_id → is_active
```

No:

```
role_id → username ❌
```

Role details are stored in Role table (correct separation).

✓ Users is in 3NF

Conclusion

Every table:

- ✓ Has atomic attributes (1NF)
- ✓ Has no partial dependency (2NF)
- ✓ Has no transitive dependency (3NF)
- ✓ Every non-key attribute depends only on the key

The database design is:

- ✓ Properly normalized
- ✓ Fully in 3NF
- ✓ Practically well-designed
- ✓ Suitable for real system implementation

Dependency diagram

SupplierID

- SupplierName
- PhoneNumber
- ContactName

ProductID

- ProductName
- SupplierID

UnitSizeID

- SizeDescription

(ProductID, UnitSizeID)

- PricePerUnit

SaleID

- SaleDate

SaleItemID

- SaleID
- ProductID
- UnitSizeID
- UnitsSold
- UnitPrice

SymptomID

- SymptomName

RoleID

- RoleName

UserID

- Username
- PasswordHash
- FullName
- RoleID
- IsActive